Scaffolding AI Context-Aware e Architettura degli Agenti

Manuale e starter kit per Claude Code e Google Antigravity / Gemini: economia dei token, gate di prontezza, routing tramite Work Order e template da copiare per avviare qualsiasi nuovo progetto. Implementazione di riferimento: github.com/f-accomando/bm (agenti con prefisso *bm-*). Nomi di file, cartelle, agenti e skill restano in inglese, così i template si copiano senza modifiche.

PARTE A — COME FUNZIONA

1. Il principio base: separare il decidere dall'eseguire

I modelli di punta (Claude Opus, Gemini Pro) hanno un collo di bottiglia di costo: l'**asimmetria dei token di output**. Un token di output di un modello top costa circa 20-40 volte quello di un modello di fascia bassa (Haiku, Flash-Lite), quindi far scrivere codice a un modello top consuma in fretta le quote di sessione. La soluzione è strutturale:

- **Modelli alti (Opus / Gemini Pro)** producono *pochi token di output*: pianificano, progettano, sciolgono i compromessi ed emettono Work Order strutturati.
- **Modelli bassi (Haiku / Gemini Flash)** producono *molti token di output*: scrivono e rifattorizzano il codice guidati da Work Order deterministici.
- **Routing per prontezza:** una task non si assegna in base alla sua grandezza o complessità apparente, ma a **quanto è già ben specificata**.

2. Workflow di sessione: zero spreco di token

Caricare documenti interi distrugge il budget di contesto e genera allucinazioni. L'architettura usa un recupero gerarchico, on-demand e chirurgico:

- **Root guardian** (`CLAUDE.md` / `GEMINI.md`): caricato all'avvio, sotto le 60 righe. Invarianti globali, regole di branch e licenza, tabella di consultazione, routing degli agenti.
- **Mappa indice** (`docs/progress.md`): l'unico indice. Non si legge per intero: serve solo a instradare verso la scheda d'area giusta.
- **Estrazione mirata** con grep e letture offset/limit: circa 100 token invece di 10.000.

Fase	Cosa fa l'AI	File coinvolti / output
1. Boot	Carica le regole base e i comandi rapidi.	`CLAUDE.md` / `GEMINI.md` (circa 50 righe)
2. Routing	Individua l'area target se non è ovvia.	`docs/progress.md` (solo consultazione)
3. Lookup	Estrae in modo chirurgico task aperte e vincoli.	`grep -n -A 15 '^## Open work' docs/progress/<area>.md`
4. Esecuzione	Ispezione mirata (indice dei simboli) e test selettivi.	Sorgenti puntuali, `make test-<area>`
5. Chiusura	Leave-no-trace: aggiorna lo stato, lancia l'audit.	`docs/progress/<area>.md`, `docs_audit.py`

3. L'architettura degli agenti: sei ruoli

Agente	Livello Claude	Livello Antigravity	Strumenti	Mandato e limiti
planner	Opus medium	Gemini Pro	Sola lettura (+ shell in sola lettura)	Trasforma task vaghe o grosse in Work Order verificati. **Non scrive mai codice.** Passa sempre prima di deep.
coder	Haiku high	Gemini Flash	Completi (edit, write, exec)	Scrive codice da un Work Order bloccato. **Nessuna scelta di design, nessuna ambiguità da sciogliere.**
mechanic	Haiku low	Gemini Flash-Lite	Completi (modifiche atomiche)	Task a rischio zero: rename di simboli, formattazione, spunte di progress/roadmap.
light	Sonnet medium	Gemini Flash	Completi (read, write, exec)	Fix mirati (1-2 file) con giudizio tecnico; documentazione verificata sul codice reale.
deep	Opus xhigh	Gemini Pro	Completi (repro isolata)	Gate rigoroso: un solo bug isolato con riproduzione, o una decisione architetturale difficile. Rifiuta le task generiche.
research	Haiku low	Subagente research nativo	Sola lettura (+ shell in sola lettura)	Esplorazione ampia su molti file; risponde con una sintesi di 5-10 righe. Non modifica mai.

4. Gate di prontezza, stati di chiusura ed escalation

Prima di assegnare una task a un agente esecutivo (mechanic, coder, light), il router risponde a cinque domande sì/no:

1. **File e simboli:** sono nominati percorsi esatti e firme delle funzioni?
2. **Determinismo del diff:** la modifica è specificata così bene che due sviluppatori scriverebbero lo stesso diff?
3. **Comando verificabile:** esiste un controllo automatico (unit test, build, lint, script di audit)?
4. **Scope limitato:** tocca 3 file o meno?
5. **Nessuna decisione sospesa:** è priva di scelte di design o dipendenze ancora aperte?

Regola: 5 su 5 SÌ = task pronta; anche un solo NO blocca l'esecuzione e la richiesta va a **planner**, che la rende pronta o la spezza in Work Order atomici. Mai una task non preparata direttamente a **deep**: accetta solo sintomo, riproduzione esatta, area sospetta e tentativi già fatti; altrimenti risponde `REJECTED` e rimanda a planner. Ogni agente chiude con uno e un solo stato:

Stato	Significato	Passo successivo
`DONE`	Il comando di verifica è passato; breve riepilogo del diff.	Chiudere la task (sezione 19, regola R9).
`BLOCKED: <motivo>`	Specifica ambigua, firma non corrispondente o decisione mancante.	Torna a planner.
`FAILED x2`	La verifica continua a fallire dopo due tentativi di correzione.	Sale di un livello (da coder a light, da light a deep).

5. Template standard del Work Order

```
### WORK ORDER: [WO-ID]
- Objective: [una frase concisa che definisce il risultato]
- Target Files & Symbols: [percorsi e funzioni espliciti, es. src/module/file.c:init_thing()]
- Required Mutation: [specifica del diff senza ambiguita', zero margine di design]
- Verification Command: [comando deterministico, es. make test-<area> oppure pytest -k <nome>]
- Expected Result: [exit code 0 e l'output/asserzione esatti attesi]
- Designated Agent: [mechanic | coder | light]
```

6. Struttura del repository su tre livelli

Per evitare il degrado del contesto, la documentazione si divide in tre cicli di vita:

- **Root guardian** (`CLAUDE.md`, `GEMINI.md`): sotto le 60 righe. Solo invarianti globali, scorciatoie di build, regole di branch, routing degli agenti. Nessuna cronologia di rilasci.
- **Progress vivo** (`docs/progress/<area>.md`): schede attive di 80-120 righe: stato attuale, lavoro aperto, regole locali non derogabili.
- **RFC profonde** (`docs/design/<topic>.md`): design speculativi e proposte a lungo termine, tenuti fuori dall'indice vivo per non gonfiare il costo in token.

PARTE B — STARTER KIT PER UN NUOVO PROGETTO (COPIA, COMPILA I <PLACEHOLDER>, COMMITTA)

7. Checklist di avvio e albero dei file

1. Crea l'albero qui sotto e copia i template delle sezioni 8-14.
2. Compila `CLAUDE.md`: stack, comandi di test, invarianti globali. Resta sotto le 60 righe.
3. Copia i sei agenti (sezione 10); adatta la lista `tools` e i comandi di verifica al tuo stack.
4. Copia le tre skill (sezione 11) e `.claude/settings.json` (sezione 12).
5. Crea `docs/progress.md` e una prima scheda d'area (sezione 13); aggiungi `scripts/docs_audit.py` (sezione 14).
6. Collega l'audit a pre-commit, Makefile e CI (sezione 15). Lancialo una volta e salva il punteggio di base.
7. Crea `GEMINI.md` se usi anche Antigravity (sezione 9). Committa, poi affida la prima task vera a planner per provare il gate.

```
<project>/
|-- CLAUDE.md                   root guardian, sotto le 60 righe (caricato a ogni sessione)
|-- GEMINI.md                   stesse sezioni 1,2,4,5 + tabella di routing Antigravity
|-- AGENTS.md                   opzionale: ruoli, gate, formato Work Order (governance condivisa)
|-- .claude/
|   |-- settings.json           skill override, permessi allow/deny
|   |-- agents/                 planner, coder, mechanic, light, deep, research (.md)
|   `-- skills/
|       |-- work-order/SKILL.md
```

```
|       |-- docs-map/SKILL.md
|       `-- docs-loop/SKILL.md
|-- .agents/skills/              copia di .claude/skills per Antigravity
|-- .githooks/pre-commit         hook versionato (git config core.hooksPath .githooks)
|-- .github/workflows/ci.yml     include lo step di docs audit
|-- docs/
|   |-- progress.md              l'unico indice
|   |-- progress/<area>.md       schede vive, 80-120 righe ciascuna
|   |-- design/<topic>.md        RFC
|   `-- ROADMAP.md               milestone opzionali
`-- scripts/
    |-- docs_audit.py            punteggio: token + penalita' (sezione 14)
    `-- cindex.py                indice opzionale di simboli/banner per file sorgente grossi
```

8. Template: CLAUDE.md (root guardian)

Claude Code concatena ogni `CLAUDE.md` che trova dalla cartella di lavoro verso l'alto e lo carica per intero a ogni sessione. La linea guida ufficiale è restare sotto le 200 righe; questo kit è volutamente più severo (60) perché il file si paga a ogni turno. È contesto, non enforcement: ciò che non deve *mai* succedere va in una regola di permesso o in un hook (sezione 12). I titoli `## Open work` e `## Rules` restano in inglese perché sono il bersaglio dei comandi grep.

`CLAUDE.md`

```
# <Project> - Note Operative di Sessione (Claude & Gemini)
Repo <org>/<repo>, branch principale `main`. Stack: <linguaggi, framework, OS/target>.
Indice unico dello stato: docs/progress.md. Mai leggere un file intero: prima grep, poi Read con offset/limit.

## 1. Recupero mirato (zero spreco di token)
| Esigenza                      | Comando                                                              |
|-------------------------------|----------------------------------------------------------------------|
| Cosa resta da fare in un'area | grep -n -A 15 '^## Open work' docs/progress/<area>.md         |
| Regole vincolanti di un'area  | grep -n -A 20 '^## Rules' docs/progress/<area>.md             |
| Trovare un simbolo in un file | python scripts/cindex.py <file> <simbolo> (poi Read offset)    |
| Contesto di design            | docs/design/<topic>.md (solo se richiesto)                     |

## 2. Regole di ingaggio
- Branch: proporre un nome breve (feat/<nome>) e chiedere prima di modificare; partire da main aggiornato.
- Test: lanciare solo i test dell'area toccata (make test-<area>); mai l'intera suite.
- Nessun segreto (chiavi, token, .env) nel repo. Commit solo su richiesta dell'utente.
- <Vincoli di licenza / dipendenze, es. nessun codice GPL copiato nei sorgenti>.

## 3. Routing dei subagent (definizioni in .claude/agents/)
Si sceglie per quanto una task e' ben specificata, non per la sua grandezza.
| Agente    | Modello / effort       | Fa                                          | Non fa                  |
|-----------|------------------------|---------------------------------------------|-------------------------|
| planner   | opus medium, sola lett.| task vaga/grossa -> Work Order              | scrivere codice         |
| coder     | haiku high             | codice da un Work Order bloccato            | scelte di design        |
| mechanic  | haiku low              | rename, format, spunte di progress          | qualsiasi decisione     |
| light     | sonnet medium          | fix su 1-2 file, docs verificate sul codice | lavori lunghi o vaghi   |
| deep      | opus xhigh             | un bug isolato con repro / una decisione    | feature intere          |
| research  | haiku low, sola lett.  | ricerca ampia, sintesi di 5-10 righe        | modificare file         |
Gate di prontezza (5/5 SI' per coder, mechanic, light): file+simboli nominati; diff deterministico; comando
di verifica; <= 3 file; nessuna decisione sospesa. Anche un solo NO -> planner. Mai direttamente a deep.
Ogni agente chiude con: DONE | BLOCKED: <motivo> | FAILED x2 (sale di un livello).

## 4. Invarianti globali
- <Invariante 1, es. l'API pubblica del modulo X e' congelata>
- Le regole di area stanno in docs/progress/<area>.md sotto "## Rules (do not break)".

## 5. Chiusura task (leave no trace)
1. Aggiornare "## Open work" nella scheda d'area (togliere o spuntare il lavoro finito).
2. Nuovo vincolo o pattern? Aggiungerlo sotto "## Rules (do not break)".
3. Lanciare python scripts/docs_audit.py score: nessun link rotto, nessun orfano, punteggio non sopra la base.
```

9. Template: GEMINI.md e AGENTS.md (parità tra agenti)

Claude Code legge `AGENTS.md` da solo **solo quando non esiste alcun CLAUDE.md** nella cartella di lavoro o sopra; se ci sono entrambi carica solo CLAUDE.md, a meno che CLAUDE.md lo importi con `@AGENTS.md` (gli import vengono espansi all'avvio, quindi non fanno risparmiare token). Non dare per scontato che AGENTS.md si carichi accanto a un CLAUDE.md. Regola di parità: `GEMINI.md` ripete alla lettera le sezioni 1, 2, 4 e 5 di CLAUDE.md e cambia solo la tabella di routing con la mappatura Antigravity qui sotto.

GEMINI.md (sezione 3 sostituita; tutte le altre sezioni identiche a CLAUDE.md)

```
# <Project> - Note Operative di Sessione (Google Antigravity / Gemini)
...sezioni 1 e 2 copiate alla lettera da CLAUDE.md...

## 3. Routing dei subagent (Antigravity invoke_subagent)
| Ruolo     | Modello        | Strumenti               | Fa / non fa                              |
|-----------|----------------|-------------------------|------------------------------------------|
| planner   | "pro"          | sola lettura            | Work Order / non scrive codice           |
| coder     | "flash"        | completi                | codice da Work Order / non fa design     |
| mechanic  | "flash_lite"   | completi, atomici       | modifiche a rischio zero / non decide    |
| light     | "flash"        | completi                | fix su 1-2 file, docs verificate         |
| deep      | "pro"          | completi                | un bug isolato con repro                 |
| research  | nativo         | sola lettura            | ricerca ampia, sintesi breve             |
Gate di prontezza, stato di chiusura ed escalation: identici a CLAUDE.md sezione 3.

...sezioni 4 e 5 copiate alla lettera da CLAUDE.md...
```

AGENTS.md (governance condivisa opzionale; si linka, non si duplica altrove)

```
# <Project> - Governance degli Agenti
1. Principio: i modelli top decidono (pochi token), quelli economici eseguono (tanti token). Routing per prontezza.
2. Tabella dei ruoli: planner, coder, mechanic, light, deep, research (modello, strumenti, mandato, limiti).
3. Gate di prontezza (5 domande) e regola: anche un solo NO -> planner.
4. Gate di ingresso di deep: sintomo + repro esatta + area sospetta + tentativi fatti, altrimenti REJECTED.
5. Stati di chiusura: DONE | BLOCKED: <motivo> | FAILED x2.
6. Formato Work Order (sezione 5 di questa guida).
```

10. Template: i sei agenti (.claude/agents/<name>.md)

Gli agenti di progetto stanno in `.claude/agents/` e prevalgono sugli agenti utente con lo stesso nome. Frontmatter obbligatorio: `name`, `description` (la description è ciò che legge il router, quindi dice *quando* usare l'agente). Campi opzionali utili: `tools` (lista consentita; omessa = eredita tutto), `disallowedTools`, `model` (`haiku`, `sonnet`, `opus`, un ID completo o `inherit`), `effort` (da `low` a `max`), `maxTurns`, `skills`, `memory`. I campi sconosciuti sono ignorati in silenzio. **Importante:** `tools` è una lista consentita, quindi un agente che deve lanciare un comando di verifica deve elencare `Bash`.

.claude/agents/planner.md

```
---
name: planner
description: Per task vaghe, grosse o con decisioni aperte: produce Work Order, mai codice. Chiamalo prima di deep.
model: opus
effort: medium
tools: Read, Grep, Glob, Bash
---
Sei l'agente di pianificazione. Il tuo output e' breve e deciso: piani e decisioni, mai codice.

## Mandato
1. Trasforma richieste vaghe o grosse in Work Order atomici e autosufficienti.
2. Nomina prima i file e i simboli esatti (grep, indice dei simboli, Read con offset/limit).
3. Manca un requisito? Non indovinare: elencalo come domanda esplicita per l'utente.

## Limiti assoluti
- Niente codice di implementazione ne' diff lunghi. Bash solo in sola lettura (grep, git log/diff, cindex).
- Output compatto: un blocco Work Order per ogni passo atomico, nel formato Work Order di AGENTS.md.
```

.claude/agents/coder.md

```
---
name: coder
description: Scrive codice solo da un Work Order preciso ed esaustivo. Nessuna scelta di design, nessuna ipotesi.
model: haiku
effort: high
tools: Read, Edit, Write, Grep, Glob, Bash
---
Scrivi codice da Work Order bloccati e non cambi nient'altro.

## Regole
1. Nessun design aperto: se l'ordine lascia una scelta aperta o manca un parametro, fermati e rispondi
   `BLOCKED: serve decisione - <dettaglio>`. Non inventare mai una soluzione.
2. Diff minimo: tocca solo i file e i simboli indicati nell'ordine. Nessun refactoring accessorio.
3. Verifica: esegui il comando di verifica dell'ordine prima di chiudere.

## Stato di chiusura (uno solo)
- DONE: verifica passata; riepilogo del diff in 2-3 righe.
- BLOCKED: <motivo>: specifica ambigua o decisione mancante; torna a planner.
- FAILED x2: ancora in errore dopo due tentativi; riporta l'errore e passa a light o deep.
```

.claude/agents/mechanic.md

```
---
name: mechanic
description: Modifiche a rischio zero gia' specificate: rename, formattazione, template, spunte di progress. Non decide.
model: haiku
effort: low
tools: Read, Edit, Write, Grep, Glob, Bash
---
Esegui task puramente meccaniche, a rischio zero e completamente specificate.

## Regole
1. Zero decisioni: fai alla lettera cio' che e' stato chiesto. Non cambiare la logica, non inventare task,
   non riordinare le priorita'.
2. Integrita': mantieni la struttura dei doc di progress; dopo un rename lancia build/lint e assicurati
   di non aver introdotto errori di sintassi.
3. Chiudi con DONE (riepilogo in 1-2 righe) o BLOCKED (istruzione ambigua o coordinate esatte mancanti).
```

.claude/agents/light.md

```
---
name: light
description: Task piccole con giudizio tecnico: domande sul codice, fix su 1-2 file, docs verificate sul codice reale.
model: sonnet
effort: medium
tools: Read, Edit, Write, Grep, Glob, Bash
---
Gestisci task mirate e documentazione verificata.

## Codice
- Il cambiamento minimo che risolve il problema; niente refactoring esteso.
- Lancia sempre il test o il comando di verifica specifico per la modifica.
- Fix ancora in errore al secondo tentativo: restituisci `FAILED x2` ed escala a deep.

## Documentazione
- Zero allucinazioni: leggi il codice reale prima di documentare un'API o un comportamento.
- Rispetta stile e formato dei doc esistenti; non toccare i sorgenti salvo commenti richiesti.

## Chiusura: DONE | BLOCKED (serve una decisione architetturale) | FAILED x2
```

.claude/agents/deep.md

```
---
name: deep
description: UN solo problema isolato e difficile con repro certa (o una decisione critica). Mai feature intere.
model: opus
effort: xhigh
tools: Read, Edit, Write, Grep, Glob, Bash
---
Sei l'agente di massima profondita': costoso, quindi intervieni solo su problemi isolati con evidenze oggettive.

## Gate di ingresso - accetta SOLO se la task contiene tutti e quattro gli elementi
1. Sintomo chiaro.  2. Riproduzione esatta e deterministica (comando, input o test che fallisce).
3. Area sospetta (file, modulo, sottosistema).  4. Tentativi gia' fatti ed esclusi.
Altrimenti rispondi: `REJECTED: rimandato a planner - mancano i requisiti di ingresso (repro / problema isolato)`.

## Mandato
- Analisi della causa radice PRIMA di toccare qualsiasi riga; poi un fix minimo e mirato.
- Implementazioni estese o riscritture tornano a coder o light tramite Work Order.
- Spiega l'evidenza che dimostra la causa prima di applicare il fix; non ripetere tentativi falliti.
- Esegui la riproduzione per certificare il fix; chiudi con DONE e un report (sintomo, causa, fix minimo).
```

.claude/agents/research.md

```
---
name: research
description: Esplorazione ampia in sola lettura su molti file; risponde con sintesi compatte. Non modifica mai.
model: haiku
effort: low
tools: Read, Grep, Glob, Bash
---
Sei il subagent di ricerca in sola lettura.

## Regole (zero spreco di token)
1. Sola lettura: nessuna modifica, nessun file nuovo. Bash solo per comandi in sola lettura (grep, git log, cindex).
2. Navigazione efficace: grep -n e indice dei simboli per le posizioni esatte; mai file enormi interi,
   usa Read con offset e limit ridotti.
3. Risposta ultra-compatta: niente blocchi di codice incollati; al massimo 5-10 punti con percorsi file:riga,
   firme delle funzioni o formati dei dati, e una conclusione di una riga per l'agente chiamante.
```

11. Template: tre skill (.claude/skills/<name>/SKILL.md)

Una skill è una cartella con un `SKILL.md`. Solo la sua **description** resta sempre nel contesto; il corpo si carica quando la skill viene invocata (progressive disclosure), quindi le description vanno tenute a una o due frasi con il trigger per primo e i corpi brevi (il tetto ufficiale è 500 righe; questo kit resta sotto le 60). Claude Code legge `.claude/skills/`; Antigravity legge `.agents/skills/`, quindi duplica la cartella se usi entrambi.

.claude/skills/work-order/SKILL.md

```
---
name: work-order
description: Costruisce Work Order e applica il gate di prontezza prima di delegare a coder, mechanic o light.
---
# Work Order e gate di prontezza
Rispondi alle 5 domande: (1) file e simboli nominati? (2) diff deterministico? (3) esiste il comando di verifica?
(4) 3 file o meno? (5) nessuna decisione sospesa?
- 5/5 SI': task PRONTA -> scegli mechanic, coder o light.
- anche un solo NO: NON PRONTA -> planner indaga ed emette uno o piu' Work Order.

## Template
### WORK ORDER: [ID]
- Objective / Target Files & Symbols / Required Mutation / Verification Command / Expected Result / Designated Agent

## Protocollo di chiusura
DONE (verifica passata) | BLOCKED: <motivo> (torna a planner) | FAILED x2 (sale di un livello)
```

.claude/skills/docs-map/SKILL.md

```
---
name: docs-map
description: Quale file aprire per area e come leggere i file grossi senza sprechi. Usala prima di doc grandi.
---
# Dove leggere e come aggiornare (poco, a sezioni)
Un solo indice: docs/progress.md. Si parte da li'. Mai un file grosso per intero: grep -n sul simbolo o su '^##',
poi Read con offset/limit. Un file gia' letto in sessione non si rilegge.

## Mappa delle aree
| Area   | Apri                       |
|--------|----------------------------|
| <area> | docs/progress/<area>.md   |

## Aggiornare i progressi dopo un cambio
1. grep -n '^##' sulla scheda d'area; leggi solo la sottosezione del modulo toccato.
2. Correggi o togli le righe diventate false. Registra lo stato (file, simboli), mai date o "abbiamo fatto X".
3. Scheda oltre ~120 righe: spezzala per tema (<area>-<topic>.md), linkala dal file madre e dall'indice.
4. Delega a mechanic se le righe sono gia' decise.

## Ricerche larghe
"Dove sta X / chi usa Y" su piu' file -> agente research (torna con la conclusione, non con i file).
```

.claude/skills/docs-loop/SKILL.md

```
---
name: docs-loop
description: Riordina CLAUDE.md e docs/progress* a giri misura-tieni-scarta. Usala dopo modifiche ai doc.
---
# docs-loop
Misura = python scripts/docs_audit.py score (piu' basso e' meglio).

## Ciclo (max 8 giri, una sola ipotesi per giro), scegli in quest'ordine
1. Una penalita' dell'audit: orfano -> linkalo; rotto -> correggi; oltre 120 righe -> spezza; duplicato -> una copia.
2. Una regola di area dentro CLAUDE.md -> spostala nella sezione "Rules" della scheda, lascia una riga.
3. Il file piu' riletto -> spezzalo o aggiungi un comando di ricerca nella skill docs-map.
4. Una riga d'indice che non serve a trovare un file -> toglila.
Ogni giro: copia di backup dei file toccati; modifica; verifica che l'informazione sia SPOSTATA, non persa; lancia score.
Punteggio sceso e nulla perso -> tieni e registra; altrimenti ripristina il backup e registra lo scarto.
Si ferma a 8 giri, dopo 2 scarti di fila, o quando finiscono le ipotesi.

## Workflow ripetuti -> skill
Un prompt o una sequenza di comandi che torna in 3 o piu' sessioni e che nessuna skill copre diventa una skill
(sotto le 60 righe, trigger nella description).

## Chiusura
Riporta punteggio prima -> dopo, giri tenuti/scartati, skill create. Nessun commit se non richiesto.
```

12. Template: .claude/settings.json

Due compiti: tenere piccolo il contesto sempre caricato e trasformare le regole che non devono mai essere violate in **enforcement** invece che in consigli. Le regole di permesso hanno la forma `Tool(pattern)`; per la logica che un pattern non esprime si usa un hook `PreToolUse`. Le impostazioni di progetto stanno in `.claude/settings.json` (condiviso); le personali in `.claude/settings.local.json` (ignorato da git). La chiave `skillOverrides` viene dal progetto di riferimento e non compare nel riferimento pubblico delle impostazioni che ho controllato: verificala sulla tua versione di Claude Code.

.claude/settings.json

```
{
  "$schema": "https://json.schemastore.org/claude-code-settings.json",
  "permissions": {
    "allow": [
      "Bash(make test-*)",
      "Bash(python3 scripts/docs_audit.py *)",
      "Bash(python3 scripts/cindex.py *)"
    ],
    "deny": [
      "Read(./.env)",
      "Read(./.env.*)",
      "Read(./secrets/**)"
    ]
  },
  "skillOverrides": {
    "anthropic-skills:docx": "off",
    "anthropic-skills:pptx": "off",
    "anthropic-skills:xlsx": "off"
  }
}
```

13. Template: indice dei progressi e scheda d'area

docs/progress.md (l'unico indice; link relativi, cosi l'audit li puo seguire)

```
# <Project> - Indice dei Progressi
Stato, non cronaca. Apri UNA scheda d'area, mai tutte.

## Which section to open
| Area   | Scheda                          | Aprila quando                      |
|--------|---------------------------------|------------------------------------|
| <area> | [core](progress/<area>.md)      | <una riga: cosa vive li'>          |

## Open milestones
- M1 <nome>: <una riga> (dettagli: [ROADMAP](ROADMAP.md))

## Reference docs
- [design/<topic>.md](design/<topic>.md): RFC speculative, non stato.
```

docs/progress/<area>.md (80-120 righe; tre heading fissi, in inglese perche' sono bersaglio di grep)

```
# <Area> - stato attuale
## How it is today
### <modulo>
- <cosa esiste adesso>; simboli da cercare: `file.ext:symbol`

## Open work
- [ ] <task>. Verifica: `<comando>`

## Rules (do not break)
- <vincolo non derogabile>. Perche': <una riga>
```

14. Starter: scripts/docs_audit.py

Un audit senza dipendenze (Python 3), provato su un progetto vuoto. Il punteggio è dato dai token sempre caricati (`CLAUDE.md` + indice) più la dimensione media delle schede più le penalità; esce con codice diverso da zero se c'è una penalità o se il punteggio sale sopra la base salvata. `score --save` scrive la base in `.docs-score`. Il codice resta in inglese.

scripts/docs_audit.py

```
#!/usr/bin/env python3
"""docs_audit.py score [--save]: one number (lower is better). Tokens = bytes / 4."""
import glob, os, re, sys
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
INDEX, SHEET_MAX, ROOT_MAX = "docs/progress.md", 120, 60
BASE = os.path.join(ROOT, ".docs-score")

def rd(p):
    try:
        return open(os.path.join(ROOT, p), encoding="utf8").read()
    except OSError:
        return ""

def tok(p):
    return len(rd(p).encode()) / 4

def links(p):
    for m in re.finditer(r"\]\(([^)#\s]+\.md)", rd(p)):
        yield os.path.normpath(os.path.join(os.path.dirname(p), m.group(1))).replace(os.sep, "/")

def score(save):
    sheets = sorted(os.path.relpath(f, ROOT).replace(os.sep, "/") for f in glob.glob(ROOT + "/docs/progress/*.md"))
    linked, pen = set(links(INDEX)), []
    if rd("CLAUDE.md").count("\n") > ROOT_MAX:
        pen.append((400, f"CLAUDE.md over {ROOT_MAX} lines: move area rules to their sheet"))
    if INDEX not in rd("CLAUDE.md"):
        pen.append((2000, "CLAUDE.md does not point to the single index"))
    for s in sheets:
        if s not in linked:
            pen.append((500, f"orphan: {s}"))
        if rd(s).count("\n") > SHEET_MAX:
            pen.append((200, f"{s} over {SHEET_MAX} lines: split by topic"))
    for f in ["CLAUDE.md", INDEX] + sheets:
        for l in links(f):
            if not os.path.exists(os.path.join(ROOT, l)):
                pen.append((300, f"broken link in {f}: {l}"))
    mean = sum(map(tok, sheets)) / max(1, len(sheets))
    total = round(tok("CLAUDE.md") + tok(INDEX) + mean + sum(p for p, _ in pen))
    print(f"score {total}")
    for _, m in pen:
        print("  !", m)
    bad = bool(pen)
    if os.path.exists(BASE) and not save:
        base = int(open(BASE).read().strip() or 0)
        if total > base:
            print(f"  ! score rose above baseline {base}")
            bad = True
    if save and not bad:
        open(BASE, "w").write(str(total))
    return 1 if bad else 0

if __name__ == "__main__":
    a = sys.argv[1:]
    sys.exit(score("--save" in a) if a[:1] == ["score"] else __doc__)
```

PARTE C — AUTOMAZIONE E STRUMENTI

15. I tre pilastri dell'automazione continua

A. Hook git pre-commit. Gli hook dentro `.git/hooks` non sono versionati, quindi l'hook sta nel repo e si indica a git una volta per ogni clone:

.githooks/pre-commit (chmod +x, poi: git config core.hooksPath .githooks)

```
#!/bin/sh
python3 scripts/docs_audit.py score || { echo "docs audit failed: fix links/orphans/size"; exit 1; }
```

B. Makefile / build tool. Aggiungi un target per i doc e includilo nel target di test standard:

```
test-docs:
    python3 scripts/docs_audit.py score    # le righe di ricetta iniziano con un TAB, non con spazi

test: test-docs   # aggiungilo ai prerequisiti del tuo target di test standard
```

C. CI/CD (GitHub Actions). Uno step dedicato dentro il workflow di build/test che gira sulle pull request:

```
      - name: Check documentation audit
        run: python3 scripts/docs_audit.py score
```

16. Focus: Google Antigravity e Gemini

Claude si appoggia a `CLAUDE.md`; Google Antigravity adotta un'architettura modulare in stile enterprise:

- **File primario `GEMINI.md` (o `AGENTS.md`):** ha precedenza assoluta. Dispone di un budget di regole dedicato (20.000 token, separato dal contesto generale) con troncamento intelligente e passaggio a puntatori on-demand se si supera il limite.
- **Progressive disclosure nativa:** le skill in `.agents/skills/` (o `.claude/skills/`) espongono nel prompt solo un nome e una description brevi; il corpo esteso si carica solo se il modello decide di attivarle.
- **Relay mobile / cloud via GitHub Actions:** con `.github/workflows/antigravity-agent.yml` l'agente può essere lanciato dallo smartphone sui runner di GitHub Actions, senza usare risorse locali.
- **Subagent paralleli asincroni** con strumenti differenziati (per esempio un agente `research` in sola lettura che tiene pulito il contesto principale, ed esecuzioni indipendenti autosufficienti).

Aspetto	Claude Code	Google Antigravity
Istruzioni di root	`CLAUDE.md` (AGENTS.md solo se manca CLAUDE.md o se importato)	`GEMINI.md` o `AGENTS.md`
Cartella delle skill	`.claude/skills/<name>/SKILL.md`	`.agents/skills/` (legge anche `.claude/skills/`)
Definizione degli agenti	`.claude/agents/*.md` (alias del modello, effort, tools)	invoke_subagent con modello "pro", "flash" o "flash_lite"
Enforcement rigido	permessi allow/deny, hook	impostazioni del runtime; tieni le stesse regole in GEMINI.md

17. Plugin per Claude: crab-ledger e better-prompt

Due plugin open source estendono efficienza e qualità delle sessioni Claude:

- **crab-ledger (monitoraggio di costi e riletture):** traccia in tempo reale l'uso dei token, conta le riletture dei file e calcola il costo di ogni sessione. Si abbina a `scripts/docs_audit.py rereads` (versione completa dell'audit) e scrive `~/.claude/crab-ledger-report.json`, usato per individuare i colli di bottiglia e i file riletti così spesso da dover essere spezzati.
- **better-prompt (prompt engineering):** raffina, struttura e ottimizza i prompt inviati a Claude: toglie le ambiguità, applica ruoli e formato chiari e riduce il consumo di token alla fonte.

Entrambi sono installabili dai rispettivi repository GitHub (crab-ledger e better-prompt). La skill docs-loop legge il report di crab-ledger per decidere quali file spezzare per primi.

18. Cosa Claude può ereditare da Antigravity / Gemini

1. **Subagent research separato per le ricerche pesanti:** invece di far leggere decine di file alla sessione principale, delega a un agente rapido in sola lettura (Haiku/Sonnet) che restituisce una sintesi di 5 righe.
2. **Progressive disclosure per le istruzioni:** sposta le procedure complesse in skill on-demand con frontmatter YAML e tieni `CLAUDE.md` al minimo indispensabile.
3. **Deduplicazione tra agenti:** tieni `CLAUDE.md` e `GEMINI.md` allineati sullo stesso indice (`docs/progress.md`), così qualsiasi membro del team può usare l'uno o l'altro assistente con le stesse garanzie.

PARTE D — REGOLE BASE PER MIGLIORARE E IMPLEMENTARE QUESTA GUIDA

19. Regole base e ciclo di miglioramento

Parti da queste regole in ogni nuovo progetto. Ognuna indica come viene fatta rispettare, così lo scaffolding migliora da solo invece di degradare.

#	Regola	Applicata da
R1	**Un solo indice.** Tutto lo stato si raggiunge da `docs/progress.md`; CLAUDE.md rimanda lì e non elenca altri doc.	Penalità audit (+2000)
R2	**Root sottile.** CLAUDE.md sotto le 60 righe; una regola che riguarda una sola area passa nella sezione Rules di quell'area, lasciando una riga.	Penalità audit (+400)
R3	**Stato, non cronaca.** Le schede dicono com'è adesso: file e simboli, niente date, niente "abbiamo fatto X". Il lavoro chiuso si toglie da Open work.	Revisione docs-loop

#	Regola	Applicata da
R4	**Schede piccole.** 80-120 righe ciascuna; quelle più grandi si spezzano per tema e si linkano da file madre e indice.	Penalità audit (+200)
R5	**Letture chirurgiche.** Mai un file grosso intero; grep poi offset/limit; mai rileggere un file già letto in sessione; le ricerche larghe vanno a research.	Report riletture di crab-ledger
R6	**Routing per prontezza, non per grandezza.** Applica il gate a 5 domande; un NO va a planner; niente arriva a deep senza repro.	Description agenti, skill work-order
R7	**Stato esplicito.** Ogni agente chiude con DONE, BLOCKED o FAILED x2; al massimo due tentativi per livello, poi sale di un livello.	Corpo degli agenti
R8	**La verifica è obbligatoria.** Nessun Work Order senza un comando deterministico; gli agenti che devono lanciarlo hanno Bash in `tools`.	Template Work Order, frontmatter agenti
R9	**Leave no trace.** Chiudi ogni task: aggiorna Open work, aggiungi ogni nuovo vincolo a Rules, lancia l'audit.	CLAUDE.md sezione 5, hook
R10	**Cricchetto.** Il punteggio dell'audit non sale mai sopra la base salvata; abbassalo di proposito e risalvalo.	docs_audit.py, CI
R11	**Ciclo di miglioramento.** Una ipotesi per giro, misura, tieni o scarta, massimo 8 giri, stop dopo 2 scarti consecutivi.	Skill docs-loop
R12	**Ripetizione = skill.** Un workflow visto in 3 o più sessioni diventa una skill sotto le 60 righe con il trigger nella description.	Skill docs-loop
R13	**Impara dal costo.** Un bug o una sorpresa costata più di una sessione aggiunge una riga alle Rules dell'area, con il motivo.	Passo di chiusura (R9)
R14	**Regola dalle evidenze.** Ogni mese conta BLOCKED e FAILED x2 per agente: molti BLOCKED = gate o planner troppo deboli; molti FAILED x2 a un livello = alza il livello o stringi il template del Work Order.	Revisione manuale
R15	**Le regole rigide si impongono, non si chiedono.** Segreti, comandi distruttivi e percorsi vietati vanno nelle regole deny dei permessi o in un hook PreToolUse, non solo in CLAUDE.md.	settings.json
R16	**Budget sempre caricato.** Spegni le skill integrate che il progetto non usa; tieni ogni description a 1-2 frasi.	settings.json, revisione
R17	**Parità.** CLAUDE.md e GEMINI.md condividono alla lettera le sezioni 1, 2, 4, 5; cambia solo la tabella di routing.	Revisione della pull request
R18	**Default sicuri.** Branch prima di modificare, solo test selettivi, nessun segreto nel repo, commit solo su richiesta.	CLAUDE.md sezione 2

PARTE E — ESEMPIO PRATICO: STIMA TEORICA VS MISURA REALE

20. Sommario dei benchmark: baseline, ottimizzazione teorica, test empirico

Due test quantificano l'effetto dello scaffolding. Il primo è una **stima analitica** su tre task rappresentative; il secondo è una **misura dal vivo** eseguita in Google Antigravity su file reali di progetto. Entrambi confrontano le stesse due pipeline.

Metodologia

- **Pipeline baseline:** un solo agente di fascia alta senza vincoli, senza routing e senza gate. Carica file interi per trovare il contesto (compreso un sorgente da ~350 KB, circa 88.000 token da solo) e scrive codice o testo direttamente.
- **Pipeline ottimizzata:** l'architettura di questa guida. Prima il readiness gate, poi lookup chirurgico (grep di una sola sezione, indice dei simboli con offset/limit), un Work Order breve dal planner, esecuzione da un agente di fascia bassa e un comando di verifica deterministico che chiude il task con uno stato esplicito.
- **Test 1, stima teorica:** tre tipi di task (un refactoring su più file, una richiesta di debug ambigua senza passi di riproduzione e un riallineamento della documentazione). Token e tempi sono stime analitiche basate sulle dimensioni dei file e sul comportamento tipico degli agenti, non misure.
- **Test 2, test empirico dal vivo:** un task di coerenza documentale (confrontare una scheda di avanzamento con le regole della sua area e validare l'integrità del repository), eseguito due volte sul repository reale: una con lettura completa dei file, una con la pipeline ottimizzata. Byte, righe, token e tempo reale sono stati misurati; la verifica era lo script di audit del progetto.
- **Metriche:** byte e righe lette, token di input, token di output (fascia alta e bassa), tempo, quota del piano gratuito consumata e se il risultato è stato verificato in modo deterministico.

Test 1 - stima teorica, totali sulle tre task

Metrica	Baseline	Ottimizzata (stima)	Diff
Token di input	252.000	2.000	-99,2%
Token di output, fascia alta	11.100	245	-97,8%
Token di output, fascia bassa	0	510	lavoro delegato
Tempo	~270 s	~22 s	-91,8% (12x)
Tasso di completamento	40% medio	100%	il gate blocca la task ambigua
Errori / regressioni	4	0	deterministico
Quota piano gratuito	esaurita al turno 2	~3,5% usata	la sessione continua

Test 2 - misura dal vivo su file reali

Metrica	Baseline (misurata)	Ottimizzata (misurata)	Diff
Byte letti	406.994	1.037	-99,7%
Righe caricate	10.039	16	-99,8%
Token di input	107.103	404	-99,6%
Token di output	578	96	-83,4%
Tempo	14,48 s	2,05 s	-85,8% (7x)
Verifica finale	nessuna	DONE, audit superato	deterministico
Quota piano gratuito	blocco sessione	0,4% usata	trascurabile

Nella baseline il singolo turno supera subito il limite di contesto del piano gratuito e la sessione si blocca. Nella pipeline ottimizzata il gate ha instradato la task al planner, un lookup di 16 righe ha prodotto un Work Order da 76 token e l'agente di fascia bassa ha lanciato lo script di audit (zero link rotti) chiudendo con **DONE** in 20 token.

Confronto a tre vie per la task di documentazione

La tabella mette a confronto baseline, ottimizzazione teorica e risultato dal vivo. La stima è stata costruita per la task di documentazione del Test 1; la baseline misurata viene dal Test 2, una task simile su un insieme di file più grande, quindi i suoi valori sono più alti della baseline stimata.

Parametro	Baseline (stima)	Baseline (misurata)	Ottimizzata (stima)	Ottimizzata (misurata)
Dati letti	documenti interi	406.994 B	~1.500 B	1.037 B
Token di input	~45.000	107.103	~450	404
Token di output	~4.100	578	~90-100	96
Tempo	~95 s	14,48 s	~6 s	2,05 s
Verifica	nessuna (prolisso, non verificato)	nessuna	audit atteso OK	audit superato, DONE
Quota piano gratuito	non eseguibile	blocco sessione	~1%	0,4%

Pipeline ottimizzata: stima vs misura

Parametro	Stima	Misura	Scostamento
Dati letti	~1.500 B	1.037 B (16 righe)	-30,8% (più snello)
Token di output, planner	~70-80	76	nel range
Token di output, mechanic	~20-30	20	nel range
Token di output, totale	~90-100	96	nel range
Token di input	~450	404	-10,2%
Tempo	~6,0 s	2,05 s	-65,8% (3x più veloce)
Esito	audit OK atteso	audit superato, DONE	confermato
Quota piano gratuito	~1%	0,4%	-60%

Come leggere i risultati

- **Il modello è accurato dove conta:** i token di output totali erano stimati in 90-100 e sono risultati 96. Letture e token di input sono usciti più bassi del previsto, quindi la stima è prudente.

- **Il tempo è il dato più largo:** la stima includeva un margine per la latenza cloud; l'esecuzione locale dei tool ha reso la prova circa 3 volte più veloce del previsto.
- **Il risparmio viene dal leggere meno, non solo da un modello più economico:** l'input è calato di circa il 99,6% grazie al gate e al lookup chirurgico. Il risparmio sull'output viene dalla pianificazione separata dall'esecuzione.
- **Limiti:** il test dal vivo copre una sola task di documentazione; i risultati di refactoring e debug ambiguo del Test 1 restano stime. I valori di quota dipendono dal piano e possono cambiare.